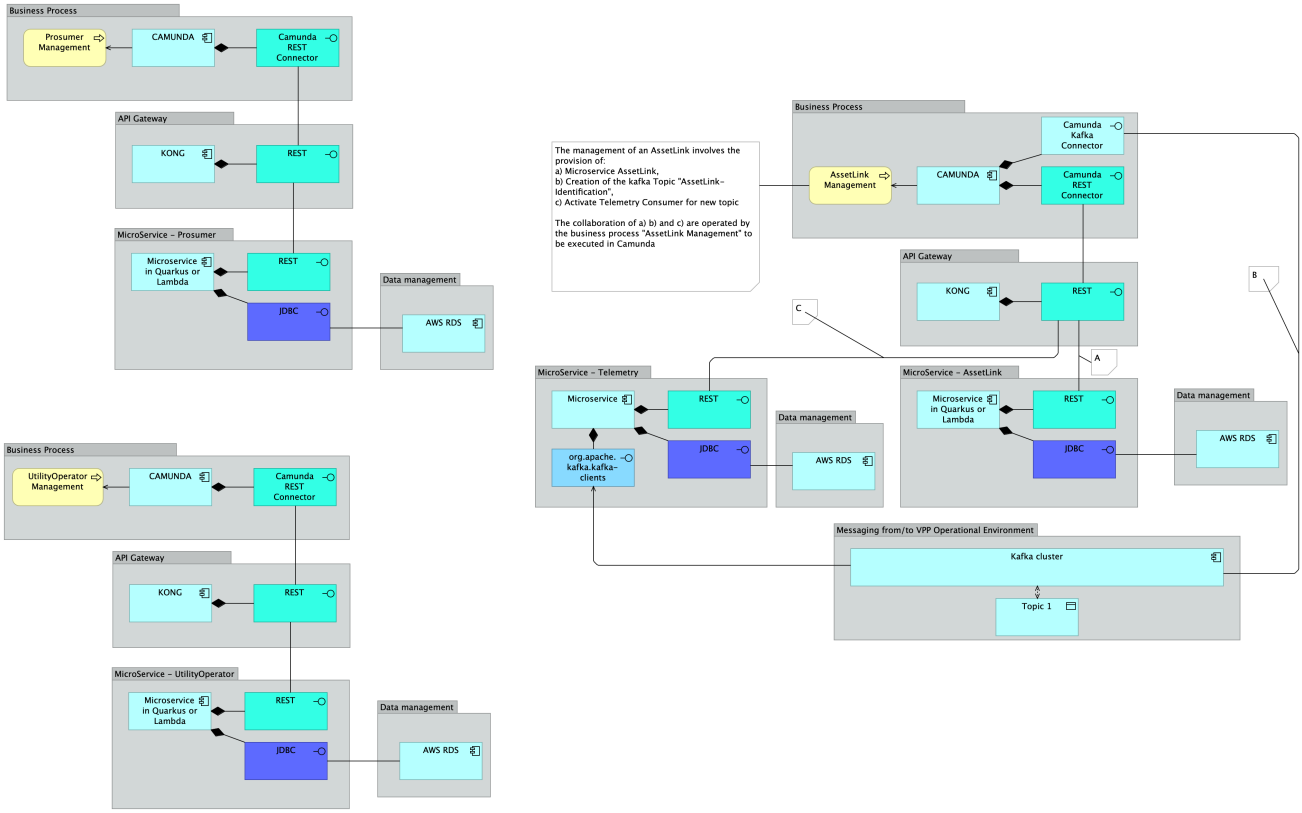
VPPaaS Proof-of-Concept (PoC)

The purpose of this (PoC) is to illustrate the design, implementation, deployment and management of all the IE technologies within the scope of your project. To that end, **three example business processes**[1] are available: Prosumer management, Utility Operator Management, and AssetLink Management. **Four example microservices** are also available[2]: Prosumer, UtilityOperator, AssetLink, and Telemetry. The application architecture is a subset of your project application architecture, as follows:



The business processes examples and microservices examples offers you a VPPaaS PoC for your study, test, reuse, and adaptation. None of the components are fully developed, neither fully traceable to all the project' requirements. You will need to understand them, and then, adapt.

To deploy this integration scenario the following plan is defined:

	Project task description	Support tool	Type of activity
0	Configure the batches with the configuration of your target environment	Vscode	Configure
1	Deploy the environment setup with the script "`DeploymentAutomation-macOS.sh`"or "`DeploymentAutomation-ubuntu.sh`"	Terraform	Deploy
2	Configure the BPMN processes configuration (service tasks) with Quarkus EC2 instance identification for microservices that are modelled in **blue** colour	Camunda modeler	Configure
3	Test the BPMN processes invoking it through Camunda modeler	Camunda modeler	Test
4	In the end undeploy all the environment using the script "`UndeploymentAutomation-all.sh`"	Terraform	Deploy

[1] For demonstration purposes, each business process is using one of the patterns provided.
[2] The required versions of Apache and Java are: Apache Maven 3.9.6 and Java version: 17. Also requires a docker hub free account.

To test it, all you must do is to change for the configuration to your own environment, at the following locations:

- **/access.sh**

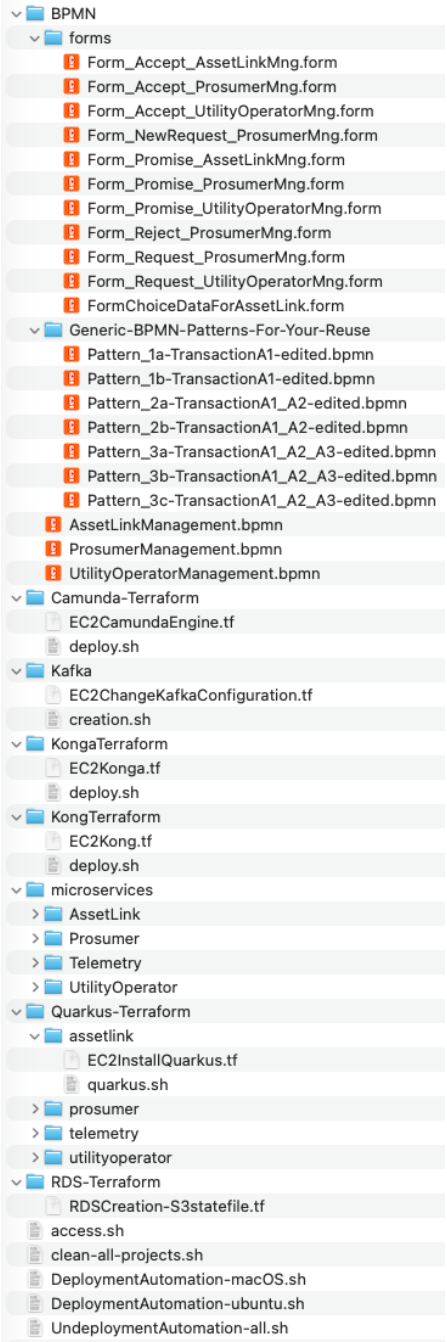
```
aws_access_key_id=YOUR AWS ACCESS KEY
aws_secret_access_key=YOUR AWS SECRET KEY
aws_session_token=YOUR AWS TOKEN
yourDockerUsername=YOUR DOCKER HUB USERNAME
yourDockerPassword=YOUR DOCKER HUB PASSWORD
```

- **/ Quarkus-Terraform/*/EC2InstallQuarkus.sh**

```
# This is an Amazon Linux ARM AMI built by Amazon Web Services - FOR DOCKER image COMPATIBILITY (check where you compiled your docker image and adapt the ami accordingly)
ami = "ami-0cd7323ab3e63805f"
```

- **Change the URL of the service tasks marked as blue in the Camunda BPMN to the correspondingly microservices endpoints**

For your learning purposes, you can find the required artifacts already available in the following folders in a zip file, accordingly with this organization:



Homework to follow-up this Integration Scenario:

- As you see in `AssetLinkManagement.bpmn` the Quarkus URL for AssetLink need to be changed whenever a change occurs in the AWS environment. How to use kong to solve this unintended effect?
  Solution: Create the routes and the services to your microservices provided address. For testing purposes, you can simplify integrating the Camunda directly to your microservices in "`DeploymentAutomation-macOS.sh`" or "`DeploymentAutomation-ubuntu.sh`"

- As you see in `ProsumerManagement.bpmn` there is a weakness of the `fiscalnumber` used as the message correlation. Therefore, if more than one instance uses the same `fiscalnumber` the solution does not work. How to solve this weakness?
  Solution: use the `ProcessInstanceKey` *from the current execution OR define a* `businessKey` *directly in the process instantiation*

- As you see in `AssetLinkManagement.bpmn` there is a weakness of the `consumerId+utilityoperatorID` used as the message correlation. Therefore, if more than one instance uses the same `consumerId+utilityoperatorID` the solution does not work. How to solve this weakness?
  Solution: use the `ProcessInstanceKey` *from the current execution OR define a* `businessKey` *directly in the process instantiation*

- How to grant Usertasks' users, groups and other accesses using the Camunda APIs?
  Solution: Use the APIs provided at: `http://<addressCamunda>:8080/swagger-ui/index.html`

- How to deploy a DMN file using Camunda APIs?
  Solution: use the endpoints provided at `http://<addressCamunda>:8080/v2/deployments`